DESENVOLVIMENTO ASSISTIDO POR IA · BLOCO 2

Geração, edição e refatoração com escopo

2

Sábado, 17/10/2026 · 9h às 12h

Ferramenta: Claude Code · modo manual por padrão

Pós-Graduação em Engenharia de Software e IA · CESAR School

CESAR

ABERTURA

Vocabulário do bloco

scope creep	escopo que cresce sem ter sido pedido
diff	diferença entre duas versões do código
plano (plan mode)	o agente propõe antes de executar
invariante	o que não pode mudar
critério de parada	quando o agente deve parar e perguntar

golden master	saídas gravadas para comparar depois
teste de caracterização	registra o que o código faz hoje
refatoração	muda a estrutura, não o comportamento
commit atômico	uma mudança coesa por commit
worktree	segunda cópia de trabalho do repositório

Ontem à noite: qual modo você escolheu, e ele foi o certo?

Responda no chat

ABERTURA

Agenda do bloco

Hoje · 17/10

9h00	10h01	10h10 às 10h40	10h40 às 11h30	11h30
Exposição: escopo, plano, refatoração	**Pausa**	**AV1.2 · rodada 1 (sem escopo)**	**AV1.2 · rodada 2 (com escopo)**	**Socialização e fechamento**

Disciplina · 6 blocos

B1 · 16/10	B2 · agora	B3 · 17/10	B4 · 23/10	B5 · 24/10	B6 · 24/10
Modos e ambiente	**Escopo e refatoração**	**Testes e docs**	**Legado**	**Revisão e segurança**	**Produtividade e time**

ESCOPO

O agente quer parecer completo, não fazer o mínimo

Sem limite, entram "melhorias" que você não pediu: renomear, trocar biblioteca, remover o que parece morto.

```
= pedido: extrair frete
+ extrai calcular_frete()
~ renomeia 14 variáveis          NÃO PEDIDO
~ troca a biblioteca de datas    NÃO PEDIDO
- remove função "sem uso"        NÃO PEDIDO
~ reformata 6 arquivos           NÃO PEDIDO
```

ESCOPO

Escopo declarado em seis campos

1

Objetivo

2

Pode tocar

3

Não pode tocar

4

Invariantes

5

Critério de pronto

6

Critério de parada

ESCOPO

Exemplo de escopo preenchido

Extrair frete de calcular_total sem mudar resultado

```
OBJETIVO     Separar desconto, frete e taxa de `calcular_total` em funções, sem mudar nenhum resultado.
PODE TOCAR   caso/miniloja/calculo.py · caso/testes_existentes/ (só adicionar)
NÃO TOCAR    Assinatura pública de `calcular_total` · outros arquivos de caso/miniloja · testes/ ·
caso/regras.md
INVARIANTES  Mesmas saídas do golden master (igualdade exata) · ordem: desconto, frete, taxa
PRONTO       `python -B testes/test_miniloja.py --codigo caso --existentes` verde · golden master verde
PARE E PERGUNTE se precisar mudar tipo numérico, tocar outro módulo ou achar algo que pareça bug
```

ESCOPO

Quando o agente encontra algo que parece bug: reporta, não corrige

PARE E PERGUNTE

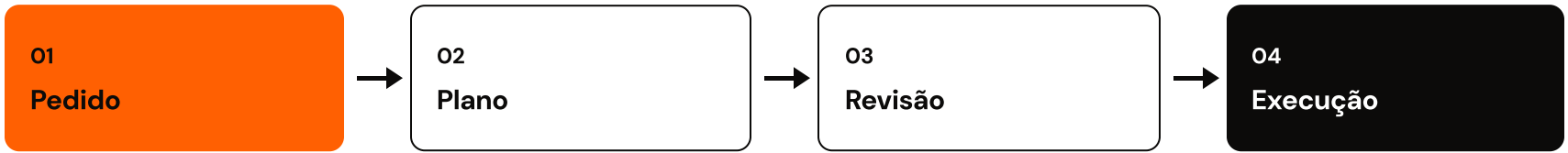
Corrigir mistura mudança de comportamento com mudança de estrutura.

PLANO

Peça o plano antes de executar

Erro caro (código errado em 12 arquivos) vira erro barato (um parágrafo errado). O plano é um artefato revisável.

No Claude Code: `Shift+Tab` até o **modo plano**. Ele só lê e propõe, e bloqueia edições até você aprovar o plano.



PLANO

Revise o plano como se revisa código

- [ ] Premissas
- [ ] Ordem dos passos
- [ ] Como verificar
- [ ] Arquivos que vai tocar
- [ ] Riscos que ele nomeou
- [ ] Onde vai parar para perguntar

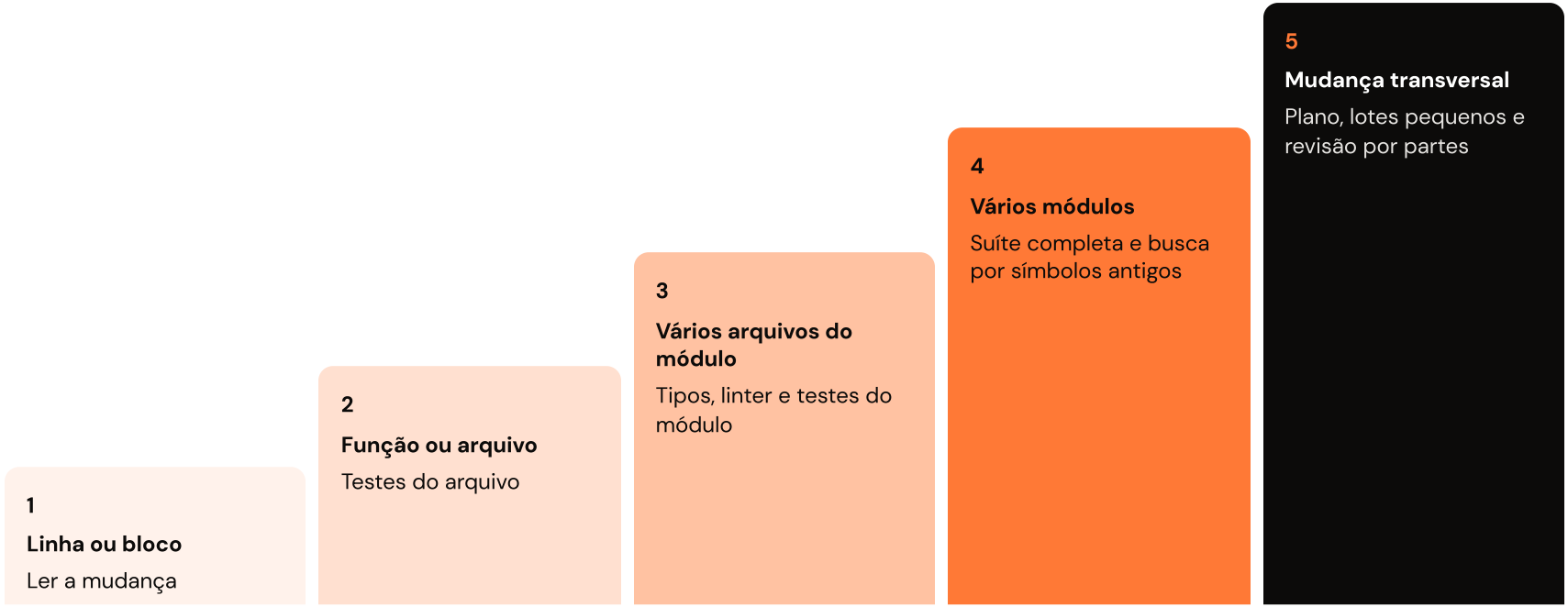
PLANO

Onde o agente deve parar e perguntar

1. Contrato público ou formato de dados
2. Dependência nova
3. Autenticação, pagamento, segredos
4. Migração de banco
5. Apagar ou editar teste
6. Passar de N arquivos
7. Requisito ambíguo

RAIO DE IMPACTO

A escada do raio de impacto



C E S A R

RAIO DE IMPACTO

Oráculos baratos para mudança multiarquivo

1

Compilar e checar tipos

2

Linter

3

Testes

4

Busca por símbolos antigos

Em linguagem dinâmica, testes pesam mais.

RAIO DE IMPACTO

Diff pequeno é decisão de engenharia

1. Capacidade de revisão
2. Localizar defeitos (bisect)
3. Reverter com segurança
4. Feedback rápido de CI
5. Lotes pequenos (DORA 2025)

REFATORAÇÃO

Refatorar x mudar comportamento: dois PRs, não um

PR 1

Refatoração

Estrutura muda, comportamento não.

PR 2

Mudança de comportamento

PR separado, com teste próprio.

Faça a mudança fácil, depois a mudança fácil.

REGRAS PERSISTENTES

Regras persistentes por ferramenta

Ferramenta	Arquivo
Claude Code	`CLAUDE.md` (não lê o `AGENTS.md`: importe com `@AGENTS.md`)
Codex e outras	`AGENTS.md`
Cursor	`.cursor/rules/*.mdc` (o `.cursorrules` é legado)
Kiro	`.kiro/steering`

REGRAS PERSISTENTES

Mais regras não é melhor

+20%

de custo com arquivos de contexto

Arquivos de contexto não melhoraram o sucesso de forma geral e aumentaram o custo em mais de 20%.

Conclusão: requisitos mínimos.

REGRAS PERSISTENTES

Um AGENTS.md mínimo e verificável

```
# AGENTS.md (mínimo e verificável)
## Comandos
- Verificações: `python -B testes/test_miniloja.py --codigo caso`
- Suíte existente: `python -B testes/test_miniloja.py --codigo caso --existentes`
## Limites
- Não alterar `testes/` nem `caso/regras.md`; não editar testes existentes para fazê-los passar.
## Convenções que o código não deixa óbvias
- O contrato é `caso/regras.md`: o esperado vem dele, não do código.
```

CLAUDE.local.md · UMA LINHA

`@AGENTS.md`

CONFERIR

Reabra a sessão e rode `/context`. No exercício: 5 a 8 linhas.

REFATORAÇÃO

Primeiro prenda o comportamento atual

Teste de caracterização: registra o que o código faz hoje, certo ou errado.

Golden master: grava saídas de entradas reais e compara depois.



REFATORAÇÃO

Golden master: igualdade exata

Sem approx: o erro clássico é trocar float por Decimal e mudar centavos.

IGUALDADE EXATA

```
# caso/testes_existentes/test_golden_calculo.py (criado por você no passo zero)
import json, unittest
from pathlib import Path
from miniloja.calculo import calcular_total

class TestGolden(unittest.TestCase):
    def test_comportamento_preservado(self):
        g = json.loads((Path(__file__).resolve().parent.parent / "golden_calculo.json").read_text(encoding="utf-
8"))
        for args, esperado in zip(g["entradas"], g["saidas"]):
            self.assertEqual(calcular_total(*args), esperado)   # igualdade EXATA
```

REFATORAÇÃO

Refatoração mecânica: editor. Decisão de design: IA

EDITOR

Renomear, mover, extrair

Refatoração automática do editor (determinística).

IA

Nomear, decidir fronteiras, simplificar condicionais

Verificada por testes.

Quando o agente insiste em reescrever o que não foi pedido

1. Reverta, não "conserte por cima"
2. Reafirme as restrições
3. Reinicie a conversa
4. Estreite a permissão de escrita
5. Aceite por partes (`git add -p`)
6. Barreira automática de escopo

Barreira automática de escopo

Falha se algo fora do escopo foi alterado ou criado.

ILUSTRATIVO: fora do repositório

```
#!/usr/bin/env bash
# scripts/scope-guard.sh
set -euo pipefail
PERMITIDOS=("caso/miniloja/calculo.py" "caso/testes_existentes/test_golden_calculo.py"
"caso/golden_calculo.json")
mudados=$( (git diff --name-only HEAD; git ls-files --others --exclude-standard) | sort -u )
for f in $mudados; do
  ok=0
  for p in "${PERMITIDOS[@]}"; do [[ "$f" == "$p" ]] && ok=1; done
  [[ $ok -eq 1 ]] || { echo "FORA DO ESCOPO: $f"; exit 1; }
done
echo "Escopo respeitado."
```

Git como rede de segurança

1. Árvore limpa
2. Branch por tarefa
3. Commit de checkpoint
4. `diff --stat` e depois o diff completo
5. `add -p`
6. worktree para comparar tentativas
7. reflog como último recurso

/rewind Desfaz só as edições das ferramentas do Claude. Não volta o que um comando de shell alterou nem edições feitas por fora: **o git continua sendo a rede de segurança**.

Janela: 10h10 às 11h30 Tempo encerrado

Passo zero e rodada 1 (sem escopo)

Passo 0	Golden master de `calcular_total` com igualdade exata, em `tarefa/golden` (com commit)
Rodada 1	Peça "refatore `calcular_total`" sem declarar escopo
Medir	Linhas, arquivos, mudança de comportamento e tempo de revisão
Desfazer	Registre `git diff --stat`, volte a `tarefa/golden` e crie `tarefa/r2`

REGRA **Se parecer bug: reporta, não corrige**

Tempo: 30 min · **Código:** `caso/miniloja/calculo.py`

ONDE

`exercicios/aula-2/README.md`
`exercicios/aula-2/template-registro.md`

```
$ python -B testes/test_miniloja.py --codigo caso
```

SEM FERRAMENTA

`apoio/respostas-simuladas/b2-rodada1-sem-escopo.md`

Janela: até 11h30 Tempo encerrado

Rodada 2 (com escopo)

1. Escreva os seis campos **antes**
2. Peça o **plano** e revise-o
3. Execute e registre as mesmas medidas
4. Escreva um `AGENTS.md` mínimo (5 a 8 linhas) e um `CLAUDE.local.md` com `@AGENTS.md`; confira com `/context`

Tempo: 40 min

ONDE

```
exercicios/aula-2/README.md
exercicios/aula-2/template-registro.md
```

```
$ python -B testes/test_miniloja.py --codigo
caso
```

SEM FERRAMENTA

```
apoio/respostas-simuladas/
b2-rodada2-com-escopo.md
```

Janela: 10h10 às 11h30

AV1.2: o que comparar entre as rodadas

Medida	Rodada 1 · sem escopo	Rodada 2 · com escopo
Tamanho do diff		
Arquivos fora do esperado		
Mudança de comportamento (golden master)		
Tempo de revisão		
Idas e vindas		

Entregável: um registro de até uma página, com o escopo, o plano que você cortou e o `AGENTS.md`

AV1.2 em salas: duas rodadas

Rodada 1	**30 min**
Rodada 2	**40 min**
Transição de salas	**5 a 8 min**

VOLTA ÀS

11h30

Tempo encerrado

CESAR

FECHAMENTO

Do prompt vago ao prompt com seis campos

RUIM **"Refatore o InvoiceService"**

BOM Objetivo, arquivos permitidos, invariantes, comando de verificação e critério de parada

```
Extraia o cálculo de taxas internacionais de `calculateTotal` para `InternationalTaxStrategy`.
PODE TOCAR   src/billing/InvoiceService.ts · src/billing/InternationalTaxStrategy.ts (novo) · tests/billing/**
NÃO TOCAR    Assinatura de `calculateTotal` · @types.ts (usar, não alterar) · package.json
INVARIANTES  Mesmas saídas do golden master, mesmas regras de arredondamento por moeda
PRONTO       `npm test` verde · golden master verde · diff < 120 linhas
PARE E PERGUNTE se precisar mudar tipo, arredondamento ou tocar outro arquivo
```

Socialização: quanto o diff encolheu? O que o plano revelado mudou?

3 a 5 grupos

FECHAMENTO

Escopo declarado antes é o que separa refatoração de reescrita acidental

Escopo | Plano revisado | Diff pequeno | Golden master

A instrução no prompt é intenção; a permissão é a trava.

CESAR

FECHAMENTO

Referências e próximos passos

AV2

Até 21/10, 23h59

Fechar a refatoração e a suíte.

`avaliacoes/av2/README.md`

PRÓXIMO BLOCO

Testes e documentação · **AV1.3, hoje às 15h10**

Feathers, Fowler, Beck. Refatoração e código legado [consolidado].

DORA (2025). State of AI-assisted Software Development. dora.dev/research/2025

Gloaguen et al. (2026). Arquivos de contexto para agentes. arXiv 2602.11988

Repositório dos alunos. github.com/jamissonfreitas/repo_alunos_d12_esia